15/08/2026

CSE-405

Cryptography

insecure comm. channel — assume some attackers is present

↳ sender & recv need to communicate securely.
others cannot read the data

- confidentiality → sender & recv should be the only ones to see the msg
- integrity → the msg should not be altered, alteration should be reported
- authenticity → recv is sure of the snc

kind of similar.

⟹ production systems should always use existing cryptographic library.

Eve → confidentiality-scenario
Mallory → integrity-scenario

integrity vs authenticity

↳ snc of msg is ensured
↳ content altered/tampered → detect

Symmetric key model ⇒ AES → same key for encryption & decryption
Alice & Bob know some shared secret key.

Asymmetric key model

everyone has 2 keys : public, private

Kerchkhoff's principle

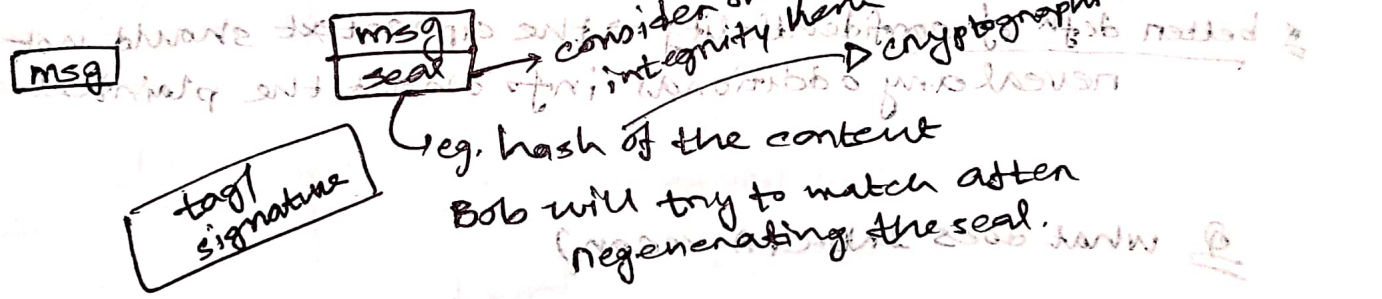
* assume everyone knows the internal details of the alg.
* the key is the only secret → single point of failure
  ↳ should be robust

effective key mgmt

destroy old key → delete all keys not in use.

128-bit → cannot be guessed Later on

① msg → plaintext → human readable
② encrypted msg → ciphertext



Threat models → possible attacks

① ciphertext-only ⇒ Attkr can only intercept
② chosen-plaintext ⇒ attkr can randomly send any plaintext and get it encrypted by Alice
③ chosen-ciphertext ⇒ get decrypted by Bob
④ chosen plaintext-ciphertext ⇒ both

↳ methods are validated against this one.

integrity → MAC → msg authentication code.

Symmetric-key Encryption

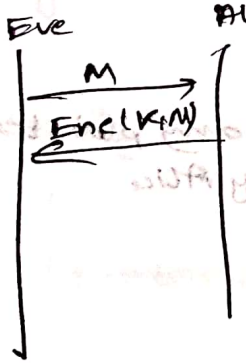
→ all msgs are represented in binary

3 algos
- KeyGen() → K
- Enc(K, M) → C
- Dec(K, C) → M

correctness ⇒ Dec(K, Enc(K, M)) = M for all K ← KeyGen() & M.

* better defn of confidentiality ⇒ the ciphertext should not reveal any additional info about the plaintext

Q what does IND-CPA mean?



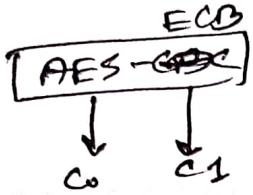
} repeat → try to analyze the encryption mechanism.

AES → ECB, CBC

P_0, P_1 → 2 pixels

(0,255,0) (0,0,255)

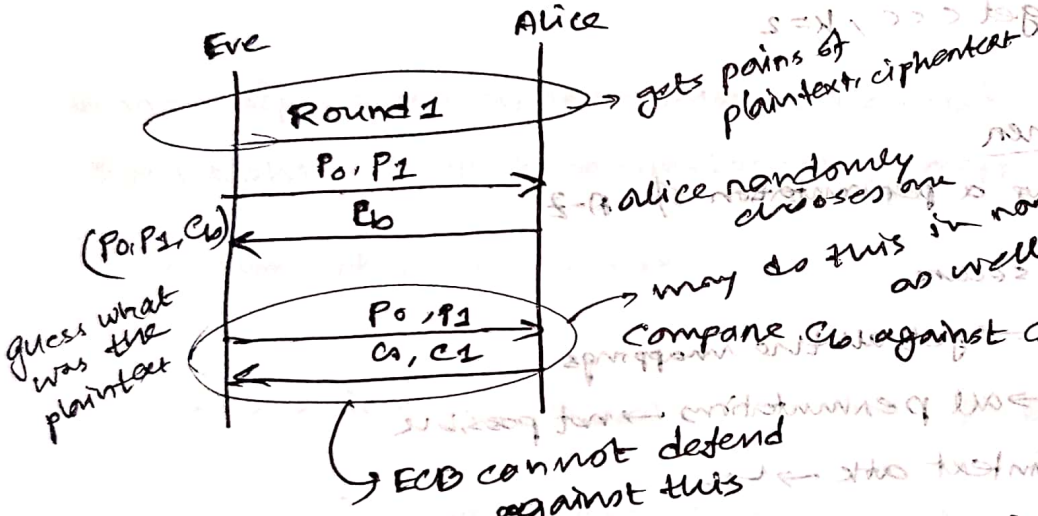
Q AES-ECB vs AES-CBC which one is IND-CPA secure.



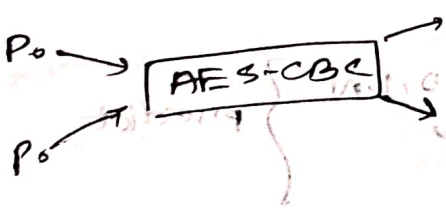
→ different if $P_0 \neq P_1$

↳ same if $P_0 = P_1$

Q Why do we need 2 rounds?



⇒ msg length is not any additional info about the msg.



proving insecurity is easier → at least one atk

⇒ Attacker runtime → limited → only polynomial atk algs

⇒ negligible advantage → eg: $1/2 + (1/2)^{128}$

↳ no better than random

current threshold $(1/2)^{80}$

$\leq 1/2 + \varepsilon$

↳ negligible.

$(1/2)^{30}$ ↳ one in a million.

Q $C \rightarrow (1/2)^{256}$

CSE-405

16/08/2026

oldest known cipher → Caesar cipher

KeyGen → 0-25 → K

Enc(K,M) → replace each letter in M with the letter K positions after the one

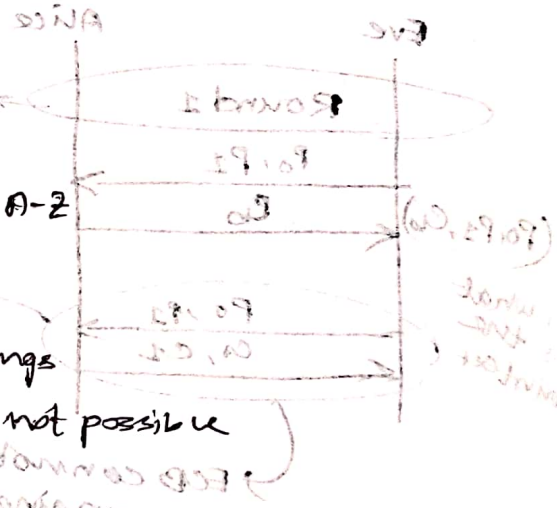
Dec(K,M) → replace with the earlier one.

not IND-CPA secure

easily predict

(i) bruteforce

(ii) send AAA, get CCC, K=2



Substitution cipher

map using a permutation of A-Z

not IND-CPA secure

send A to Z → get all the mappings

(i) bruteforce → all permutations → not possible

(ii) chosen-plaintext atk → ~~C~~

Cryptanalysis

common letters → E, T, A, O, I, N

find freq in ciphertext → } predict

Enigma → WWII → optional.

significant weakness → a character is never mapped to itself

↳ exploit this



long enough sequence → only one letter is left out

B won't occur here
↳ mapping of M1 → beats IND-CPA

∴ given an encryption scheme & its weakness how can we beat the IND-CPA game?

* mono alphabetic cipher ⇒ substitution cipher with one-to-one mapping
* polyalphabetic cipher ⇒ one-to-many mapping → eg. Enigma code.

caesar cipher → additive

∴ diagram of the scheme may be given

K=20 → decryption ⇒ additive inverse of 20 in Z_{26} ⇒ 6

Z_{26} : modulo group 26

(i) closure
(ii)
(iii)
(iv)
} properties of a group.

group defined under addition operation

$\langle Z_{26}, + \rangle$ → under addition

{0, ... 25}

closure ⇒ (1) add any num from [0-25] with any one & mod 26 → in [0,25]

(2) 6+(-6) → 0 ⇒ additive → -6 not here

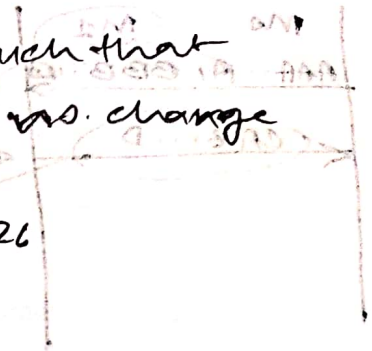
6 + (20) mod 26

20 is the additive inverse of 6 under modulo 26 group

③ identity property : $(a+0) \bmod 26 = a$

some num exists in [0,25] such that adding it results in no change

$(a+b)+c \bmod 26 \equiv a+(b+c) \bmod 26$



multiplicative cipher

$C = (P \times K) \bmod 26$; K^{-1} ⇒ multiplicative inverse of K

$P = (C \times K^{-1}) \bmod 26$

$a \boxed{b} \bmod 26 \equiv 1$

↳ multiplicative inverse of a

$a=3 \rightarrow b=9$

algo

check if gcd of 3 and 26 = 1 → else multiplicative inverse doesn't exist.

① gcd (3, 26) == 1 ?

② solve the linear eqn ⇒ $3(x) + 26y = 1$ ← diophantine eqn.

↳ multiplicative inverse

3	26	8
	24	

26y → doesn't change the result of modulo

$26 = 3(8) + 2 \rightarrow 2 = 26(1) + 3(-8)$

$3 = 2(1) + 1 \rightarrow 1 = 3(1) + 2(-1)$

backward substitution

$= 3(1) + 26(-1) + 3(8)$

$1 = 3(9) + 26(-1)$

multiplicative inverse of 3

15 ① $\gcd(15, 26) = 1$

② $15x + 26y = 1$

② $26 = 15(1) + 11 \rightarrow$ ⑪ $= 26(1) + 15(-1)$

$15 = 11(1) + 4 \rightarrow$ ④ $= 15(1) + 11(-1)$

$11 = 4(2) + 3 \rightarrow$ ③ $= 11(1) + 4(-2)$

④ $4 = 3(1) + 1 \rightarrow 1 = 4(1) + 3(-1)$

$1 = 4(1) + 3(-1)$

$= 4(1) + 11(-1) + 4(2)$

$= 4(3) + 11(-1)$

$= 11(-1) + 15(3) + 11(-3)$

$= 15(3) + 11(-4)$

$= 15(3) + 26(-4) + 15(4)$

$= 15(7) + 26(-4)$

$\rightarrow$ ⑦

$26 = 15(1) + 11$

$15 = 11(1) + 4$

$11 = 4(2) + 3$

$4 = 3(1) + 1$

$1 = 4(1) + 3(-1)$

$1 = 4(3) + 11(-1)$

$1 = 15(3) + 11(-4)$

$1 = 15(7) + 26(-4)$

$x = 7$

additive, multiplicative combined → 2 shared keys

k_1, k_2

$c = (p \cdot k_1 + k_2) \bmod 26$

2 vars. → consider these.

Affine cipher

↳ test TF

17/08/2026

additive cipher → decrypt ⇒ $p = (c - k) \bmod 26$

sub → additive inverse is added.

multiplicative → limitation → single key

if atkr gets a (p, c) pair & knows the scheme, may recover k

Hill cipher

↳ key → n×n matrix

$$\begin{bmatrix} c_1 \\ c_2 \\ \vdots \\ c_n \end{bmatrix} = \begin{bmatrix} k_{11} & \cdots & k_{1n} \\ k_{21} & \cdots & k_{2n} \\ \vdots & & \\ k_{n1} & \cdots & k_{nn} \end{bmatrix} \begin{bmatrix} p_1 \\ p_2 \\ \vdots \\ p_n \end{bmatrix} \pmod{26}$$

longer plaintext → multiple col

k → row major
p → col major
} textbook

$c = KP \bmod 26$

$P = K^{-1} C \bmod 26$

↓

modular matrix inverse

K = CDDG → given, generally fits a square matrix.

P = ATTACK

↓ col major

$\begin{bmatrix} A & T \\ T & A \\ C & K \end{bmatrix}$

$\begin{bmatrix} C & D \\ D & G \end{bmatrix}$ row major

$\begin{bmatrix} 2 & 3 \\ 3 & 6 \end{bmatrix}$

$\begin{bmatrix} 0 & 19 & 2 \\ 19 & 0 & 10 \end{bmatrix}$

5 letters → some padding scheme will be mentioned eg use Z.

$C = KP \bmod 26 = \begin{bmatrix} 2 & 3 \\ 3 & 6 \end{bmatrix}\begin{bmatrix} 0 & 19 & 2 \\ 19 & 0 & 10 \end{bmatrix}$

$= \begin{bmatrix} 57 & 38 & 34 \\ 114 & 57 & 66 \end{bmatrix} \bmod 26 = \begin{bmatrix} 5 & 12 & 8 \\ 10 & 5 & 14 \end{bmatrix}$

Q → find the ciphertext.

$= \begin{bmatrix} F & M & I \\ K & F & O \end{bmatrix}$ (col major)

Ciphertext: FKMFIO

$P = K^{-1}C \bmod 26$

$K^{-1} = \frac{Adj(K)}{|K|} = |K|^{-1} \times Adj(K)$

$|K| = 3, \quad |K|^{-1} = 9$

$3 \times 9 \bmod 26 \equiv 1$ (Euclidean algo)

$Adj(K) = \begin{bmatrix} 6 & -3 \\ -3 & 2 \end{bmatrix}$

-ve → keep adding 26, until it becomes positive

$K^{-1} = \begin{bmatrix} 54 & -27 \\ -27 & 18 \end{bmatrix} \bmod 26$

$= \begin{bmatrix} 2 & 25 \\ 25 & 18 \end{bmatrix}$

$$P = \begin{bmatrix} 2 & 25 \\ 25 & 18 \end{bmatrix} \begin{bmatrix} 5 & 12 & 8 \\ 10 & 5 & 14 \end{bmatrix} \bmod 26$$

$$= \begin{bmatrix} 260 & 149 & 366 \\ 305 & 390 & 452 \end{bmatrix} \bmod 26$$

$$= \begin{bmatrix} 0 & 19 & 2 \\ 19 & 0 & 10 \end{bmatrix}$$

ATTACK

Q given P, K, find C

Q $K = \begin{bmatrix} k_1 & 3 \\ 3 & k_2 \end{bmatrix}$, P, C given, find k_1, k_2

↳ system of eqns.

hill cipher → key matrix → still may find the key if they have a number of (P, C) pairs.

additive, multiplicative, hill → no randomness.

same msg will always results in the same ciphertext.

↳ attkr understands that the same msg is being sent

↳ leaks info about the msg.

⊗ to be IND-CPA secure, randomness is necessary, but not sufficient.

Diffusion → make the relation b/w P & C complex

P → C

Large paragraph → change one bit → the ciphertext should be entirely different from the original one.

→ this ensures that given many (P,C) pairs, the attkr cannot find the key.

Confusion → make the relⁿ b/w P & K complex

one bit change in K, entirely different C

till now → pen & paper ciphers.

One-Time Pads → XOR

Key Gen:

msg ⇒ n bit

XOR it with n bit random-key → randomly chosen bitstring

symmetric → Alice & Bob both know the key somehow

* encrypt → XOR } if K is purely random, C will be purely random

⇒ if K lacks randomness, C will leak info about P.

* decrypt → XOR

the next msg block will use a diff random bitstring.

↳ one-time use.

⇒ new key for every msg

Correctness $Enc(K,M) = K \oplus M$

$Dec(K, Enc(K,M)) = Dec(K, K \oplus M)$

$= K \oplus (K \oplus M)$

$= M$

AES → can also prove its correctness

↳ length

security

Eve → sends m_0, m_1

Alice → sends $Enc(k, m_0)$ or $Enc(k, m_1)$

$c = k \oplus m_0$ $\quad$ $c = k \oplus m_1$

$\Rightarrow k = c \oplus m_0$ $\quad$ $\Rightarrow k = c \oplus m_1$

recovered

but Eve does not know whether m_0 was returned or m_1

↳ no new info. → perfectly secure scheme.

2-time pad - use the same key twice

$m_0 \rightarrow k \oplus m_0$

$m_1 \rightarrow k \oplus m_1$ } read these → attkr can get to know whether m_0 and m_1 are same or not

m_0 | 0 | 1 | 0 | 0 | 1 |

m_1 | 0 | 0 | 1 | 0 | 1 |

$(k \oplus m_0) \oplus (k \oplus m_1)$

$m_0 \oplus m_1$

| 0 | 1 | 1 | 0 | 0 |

→ same bit → 0

↳ partial info about m_0, m_1 leaked.

→ knowing one of these, the other can be found.

Hence, use the key once. and then discard.

Impracticality

① key gen → expensive → randomness is costly

AES-CBC → only generate IV once & use it as a seed.

② key distribution → hard to share the key? → not possible

↳ communicate keys in advance

eg: in-person

but without a prior secure channel, this is not possible.

Block cipher

one to one mapping

Enc → maps n-bit P to a random permutation of n-bits depending on the k-bits key.

$E_k: \{0,1\}^k \times \{0,1\}^n \rightarrow \{0,1\}^n$

$D_k: \{0,1\}^k \times \{0,1\}^n \rightarrow \{0,1\}^n$ → inverse

↓

E_k needs to be one-to-one

bijective

↳ else cannot decrypt.

many-to-one

↳ can't decrypt

AES → 4 sub-blocks → all are bijective.

* if the E_k is random enough, C can't be differentiated from a random text → secure

* brute-force atk → not possible.

US military → AES-512

AES-256 → post quantum cryptography.

CSE-405

22/08/2026

issues
one-time pad ⇒ key generation is expensive ①
↳ if the same key is used, two msgs can be used to cancel out the keys ②

block cipher

$\{0,1\}^k \times \{0,1\}^n \rightarrow \{0,1\}^n$

2^n 2^n 2^{2n} mappings possible

once the plaintext is picked → 2^n possible ciphertexts

128-bit P, 64-bit K → X
256 → redundant

XOR, bit-shifting, small table lookups → fast on modern processors

DES → used before AES
Blk → 64 bits, K → 56 bits

AES k = 128, 192, 256
blocksize → always 128 bits

diagram → unlikely to appear in the exam

Q ① Why multiple rounds?
⇒ confusion & diffusion
② which parts help introduce/increase confusion & diffusion?

vanilla AES → deterministic

block ⇒ not IND-CPA secure ①
M_0/M_1 → get one back
next send back one at a time & match.

② fixed sized encrypted msg.

⇒ to tackle these different modes of operation have been introduced.

ECB/CBC

* longer msg ⇒ multiple blocks → concat the blocks later on

drawback → electronic code → no randomness,
↳ same P, K → same C
↳ not IND-CPA secure.

(IV) → initialization vector → random → generated once
random number generation → costly.

⇒ good algo → C is indistinguishable from a random number.
↳ use C as the next random factor.

↳ similar ideas are applied in many cases.

⇒ CBC

$C_1 = E(P_1 \oplus IV)$

$C_i = E(P_i \oplus C_{i-1})$

$D(C_1) = P_1 \oplus IV$

$D(C_1) \oplus IV = P_1$

encryption has dependencies
↳ cannot parallelize this

⇒ decryption can be parallelized.

recv has key,
send (IV) $C_1, \ldots, C_m$
↳ random num

attacker can't do anything with this

Q derive decryption eqn from encryption eqn.

Q decrypt diagram from encrypt diagram

Q diagram → which one can be parallelized?

* if msg length is not a multiple of (n) → pad

multiple schemes:

① 100--

② PKCS #7

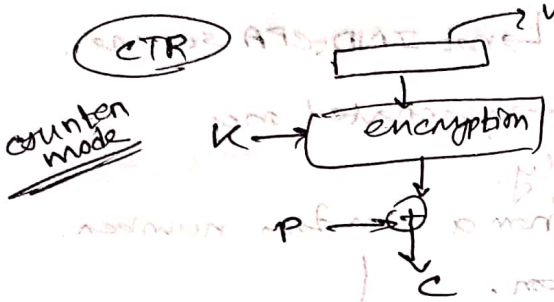
* AES-CBC ⇒ IND-CPA secure; IV is not reused.

if the IV is reused ⇒ leaks info about how long the plaintext stays identical.

$P_1 P_2 P_3$ $P_1 P_2 P_4$

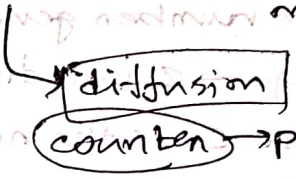
same P_i → same C
same IV

drawback → slow encryption

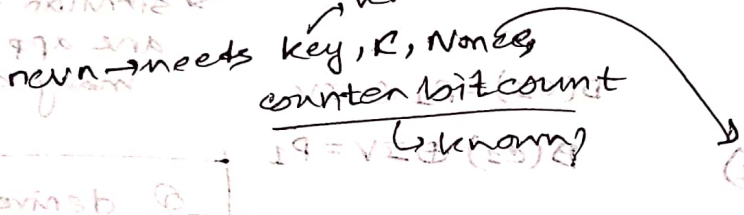


use the same for all ?
↳ use same P → same C.

diff random nums → differ in one position



Nonce → number once

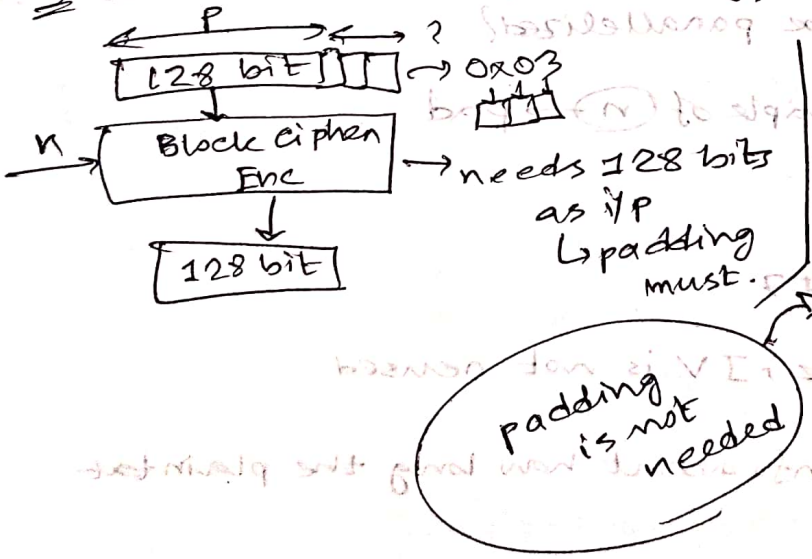


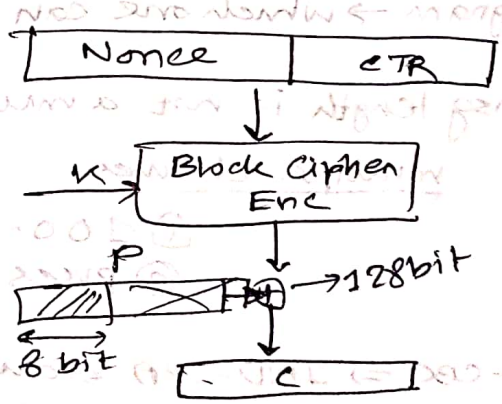
~~C = E(K) ⊕ P~~

decryption ⇒ switch the position of P & C.

Q diagram → encrypt/decrypt → check P & C positions.

Q do we need to pad the msg?



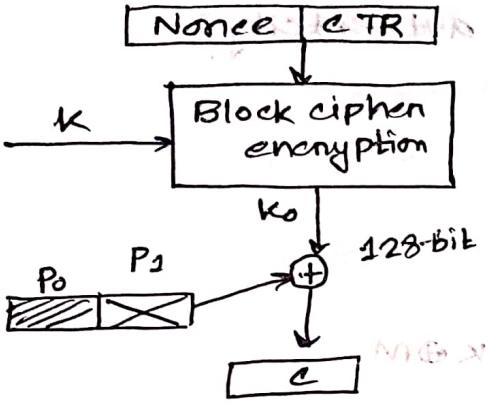


just XOR the leftmost 8 bits.

IND-CPA secure, Nonce random, not reused.

CSE-405

22/08/2026



$P_0 \oplus K_0$
$P_1 \oplus K_0$ → if Nonce is reused

info → which positions are identical?

CBC ⇒ only reveals whether the blocks are identical
↳ less damaging

⊛ high performance → CTR ⇒ both encryption and decryption can be parallelized.
↳ if used properly

⊛ reuse → risk → CBC is better → less damaging.

23/08/2026

CTR ? IND-CPA

$2^{n/2}$ blocks encrypted at once → birthday attack

CFB ⇒ feedback

⊛ pad?
⊛ consequences of erroneous IV.
⊛ only decryption can be parallelized.

⊛ how do we ensure integrity & authenticity?

Mallory → manipulator

↳ has known (P,C) pair.

reverse engineer $E_K(i)$

$c = 0x058$

$K = 0x69$ ← unknown to Mallory

$M = 0x31 \rightarrow ①$

$c = K \oplus M$

$K = c \oplus M = 0x69$

~~$M = 0x58$~~ $M = 0x39 \rightarrow ②$

~~change both~~ change $c \rightarrow K \oplus M$

↓

decrypt

↓

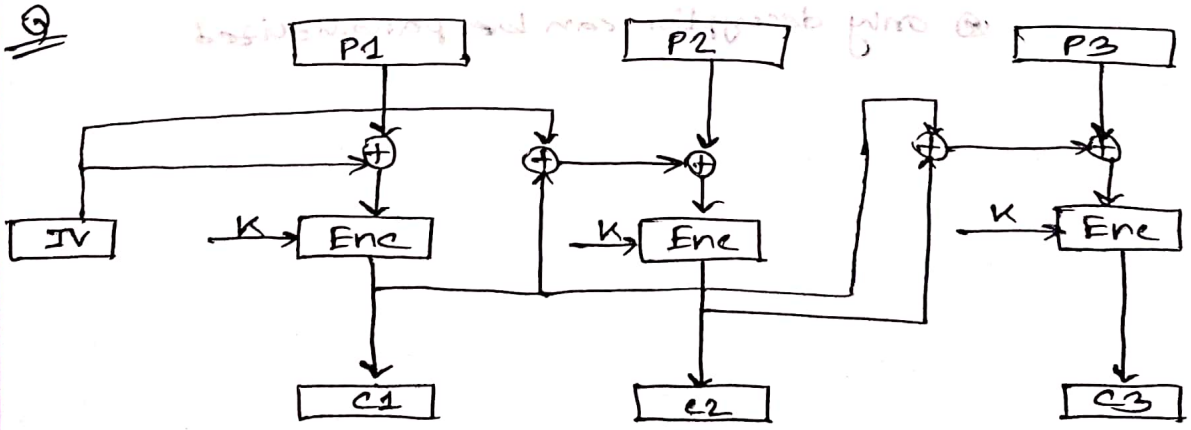
wrong msg.

⊛ Mallory only needed to know one (P,C) pair.

CBC ⇒ change one bit ⓒ → everything becomes garbage but Bob cannot determine whether Alice actually sent this msg.

⊛ Block cipher cannot ensure integrity & authenticity.

②



cannot be parallelized — can be parallelized.

$C_1 = E(P_1 \oplus IV) \longrightarrow P_1 = D(C_1) \oplus IV$

$C_2 = E(P_2 \oplus C_1 \oplus IV) \longrightarrow P_2 = D(C_2) \oplus C_1 \oplus IV$

$C_3 = E(P_3 \oplus C_2 \oplus C_1) \longrightarrow P_3 = D(C_3) \oplus C_2 \oplus C_1$

---- decryption block diagram.

Mallory → $(IV, C_1, C_2, C_3) \rightarrow (IV, C_2, C_1, C_3)$
intercepted reordered.

similar problems

Bob receives (IV, C_2, C_1, C_3)

cs161 → website practice problems.

$P_1' = D(C_2) \oplus IV \longrightarrow P_2 \oplus C_1$

$P_2' = D(C_1) \oplus C_2 \oplus IV \longrightarrow P_1 \oplus C_2$

$P_3' = D(C_3) \oplus C_1 \oplus C_2 \longrightarrow P_3$

Cryptographic Hash & MACs

cryptographic hash function ⇒ arbitrary length input, fixed output

$\{0,1\}^* \rightarrow \{0,1\}^n$

↳ must be deterministic

* connectedness
* efficiency
* security → one-way-ness ⇒ reverse-engineering (compute heavy)
* collision resistance → $M_1, M_2 \Rightarrow$ same hash
* unpredictability → one bit change in M
  ↳ totally different hash
  ↳ similar to diffusion

security

↳ random oracle assumption

one-way-ness ⇒ polynomial time adversary

$y = H(x)$

given y, adversary cannot find $x' \Rightarrow H(x') = y$ (any x')

↳ negligible probability

* $H(x) = 1$ ⟶ not one-way ⇒ any i/p yields 1.

one-way function but not one-way hash.

* collision-resistance ⇒ $x \neq x'$ and $H(x) = H(x')$

↳ not possible to design such a fune

⇒ pigeonhole principle. arbitrary → fixed

⇒ finding collisions should be infeasible

* n-bit output requires $2^{n/2}$ tries on avg. birthday attack

MD5 → broken → 128 bits

SHA-1 → 160 bits → broken in 2017, still in use

SHA-2 ⇒ 256, 384, 512

↳ extension attks possible ⟶ length extension attack

but reversal is still not possible / x' such that $h(x') = y$.

SHA-3 → diff. standard.

SHA ⇒ secure hash algorithm

Length extension attacks

orig msg needn't be known, only the length is needed.

Given H(x) and length(x) → attkr can create H(x||m) for any m.

→ why is this a problem?

x m'

url params ⇒ '&amount=1000 & user=alice' & user=mallory

H(x) = 765cad...
len(x) = 42 → optional actually → H(x||m') → this can be determined

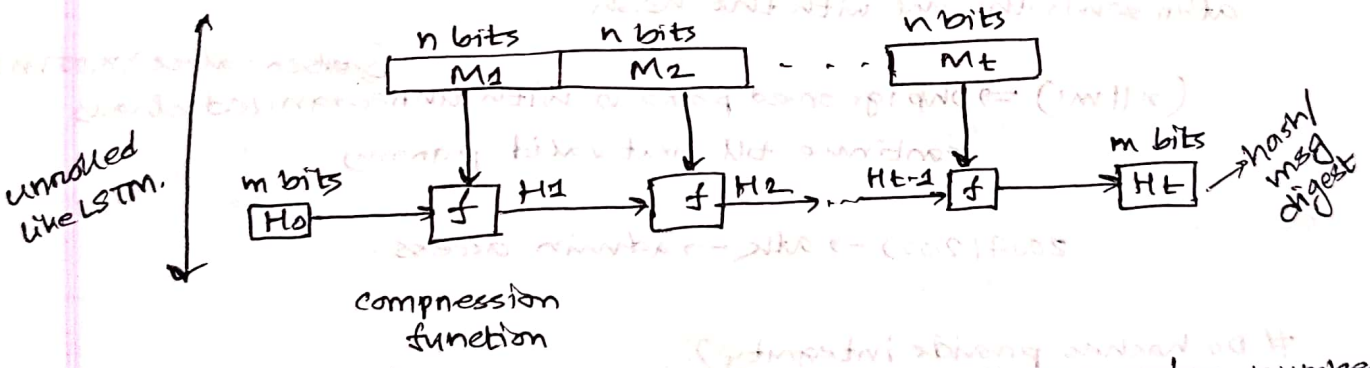
older versions

php ⇒ same parameter used again → overwritten
alice is overwritten by mallory

⇒ popular cryptographic hash functions use a chaining strategy

Merkle-Damgard construction

SHA-256 ⇒ 256 bit blocks → padding is used.



compression function

↳ i/p: prev o/p ⇒ initial value? → random number, H0 (similar to IV)

m < n

AES → IV must not be reused
hash/SHA → H0 → must be fixed else the func won't be deterministic.

H(x|m)

↳ use H_t in place of H_0
use m in place of x
} compute the new hash value w/o knowing the original msg.

* if m' needs to be used as payload, the length is needed to know about the padding ⇒ hence the length is required.

&amount=1000&user=alice\x00\x00 &user=mallory\x00\x00
x padding m'

765..

server would ignore the padding from x.

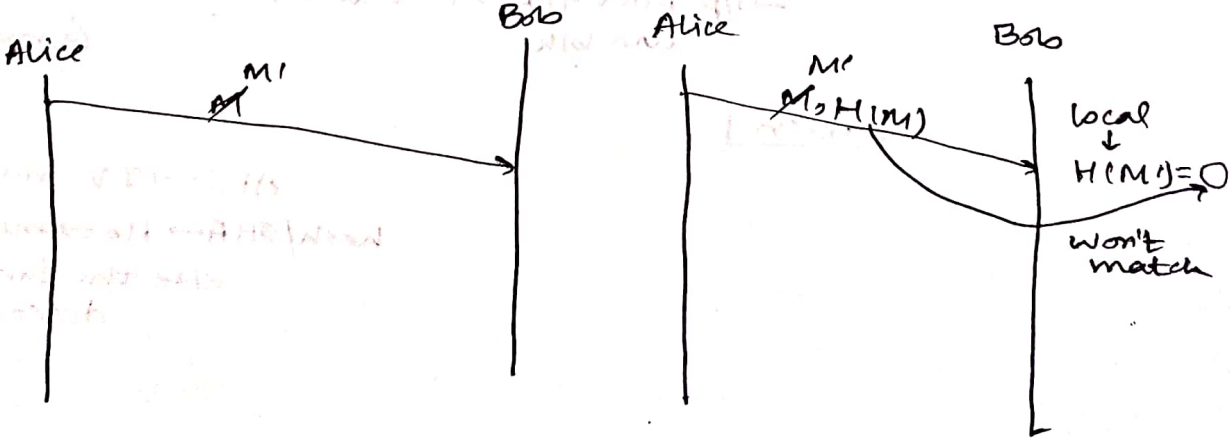
attn sends the url with the hash

(x||m') ⇒ php ignored params with unrecognized chars continue till next valid param

~~&user=alice\x00\x00~~

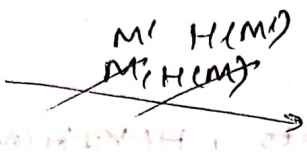
2007/2009 → atk → admin access..

Do hashes provide integrity?



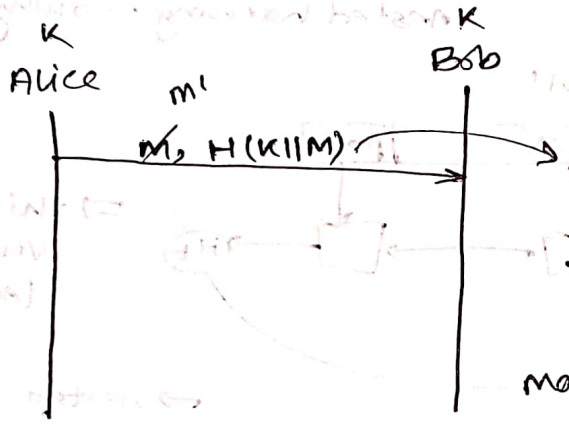
problem ⇒ attkr can modify the hash as well. → M', H(M')

problem → anyone can compute the hash for any msg.



↳ unkeyed func

∴ plain and simple hash functions do not provide integrity.



K → shared secret key

the key is concatenated

Message authentication code (MAC)

Mallory cannot compute H(K||M')

tag, T = MAC(K,M) ⇒ non-deterministic hashes exist

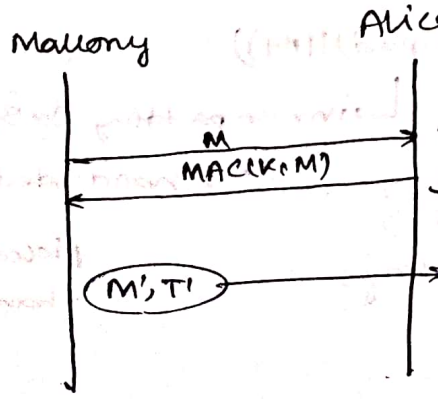
MAC
↳ KeyGen()
↳ MAC(K,M)

* deterministic
* efficient
* EU-CPA
  ↳ existentially unforgeable

non-forgeable ⇒ [w/o K, MAC(K,M') cannot be generated]

M' ≠ M ⇒ MAC(K,M') = MAC(K,M)
↳ non-existential

CPA → chosen plaintext attack



repeat → try to guess how the tag is computed

M' is not one of the msgs used here

if T' is the actual tag → not EU-CPA secure

Prob of getting the correct tag should be ~~almost~~ at most 0 or negligible

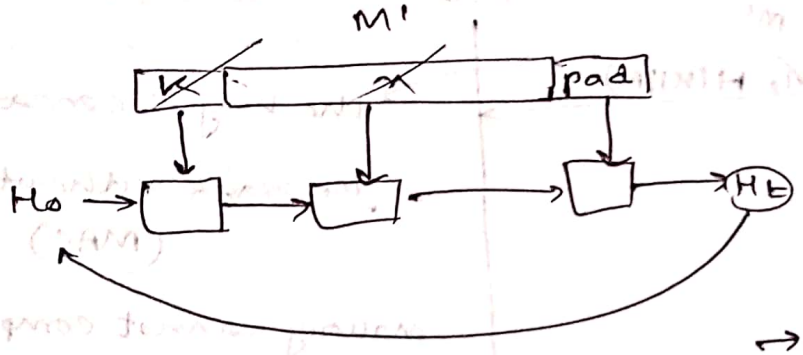
$\approx \frac{1}{2^{128}}$

NMAC

KeyGen() → k_1, k_2 → n-bits ; $H(k_1 \| H(M))$ → n-bits

$NMAC(k_1, k_2, M) \Rightarrow H(k_1 \| H(k_2 \| M))$

nested hashing → why?



⇒ this stays vulnerable to length extension attack.

→ nested hashing solves this issue.

length extension attack on $H(k_1 \| H(k_2 \| M))$

↳ can manipulate/extend this → futile

* cannot extend M anymore

Issues

2 keys → generation ...

keys must be exactly n bits.

arbitrary → not specified.

HMAC(K, M)

$H((K' \oplus opad) \| H((K' \oplus ipad) \| M))$

↳ inner padding (0x36) → hardcoded constant

↳ outer padding (0x5C)

handles shorten/lengthen keys

picked using Shannon's information theorem.

⇒ MAC provides integrity

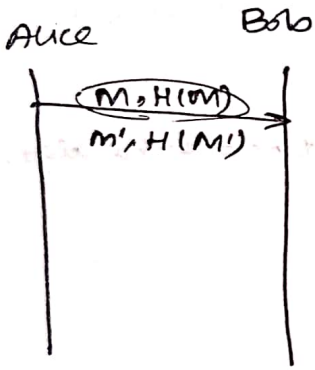
→ authenticity ⇒ depends

only 2 users: A, B → Yes

multiple users: how to distinguish

→ confidentiality → ✗ , deterministic, IND-CPA secure → ✗

29/08/2024



if mallory changes both, Bob can't distinguish

Burp suite ⇒ tool

chrome

[username: 2105123
pass: ....] HTTP → Bwet

intercept

https://...

Lock
↳ everything sent is encrypted
↳ decryption at the server.

Burp suite
↳ takes the req from chrome, decrypts it & sends to Bwet

certificate
↳ trusted

[encrypted using public key of Burp suite]

intercept & change everything → MitM Attack
Man in the Middle

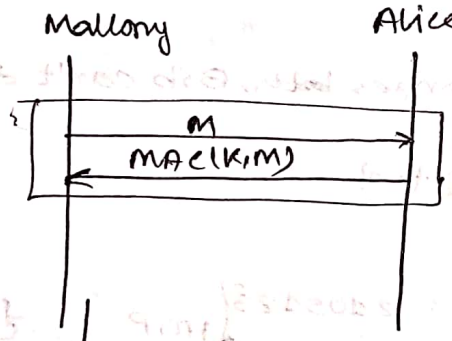
MAC

verify → separate the tag
locally compute and match.

algo ⇒ fixed output size ⇒ always the last (k) bits → Tag
easily separable

EU-CPA

w/o the key, the attacker can't generate the valid tag.



repeat → try to understand the algo.

output (M', T') → winning prob should be 0.

NMAC → 2 keys

length extension → possible to do this on $H(K_2 \| H(K_1 \| M))$
not M.

problem
↳ (2) keys
↳ same length.

used nowadays → HMAC
↳ padding

Authenticated Encryption ⇒ confidentiality + integrity

↳ ② ways
  ↳ combine schemes → we will focus on this
  ↳ new scheme

Alice sends Enc(K_1, M) and MAC(K_2, M)

Enc(K_1, M) → AES-CBC
MAC(K_2, M) → HMAC

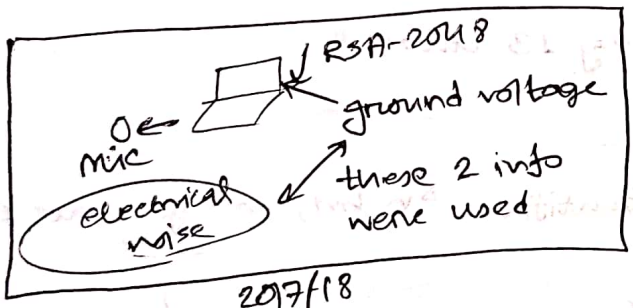
match

problems:

① Bob needs to decrypt first → M → MAC(K_2, M)

Decryption oracle → Bob must do this
  → msg may be from the attacker

anything the server must do
  ↳ opportunity for side-channel-attack
    processing time



2017/18
breaking RSA with microphone - - paper.

② MAC → plaintext
  ↳ same msg → same MAC
    ↳ leaks info about the msg.

Encrypt-then-MAC

✅ Enc(K1, M), MAC(K2, Enc(K1, M))
↳ solves both problems.
↳ no decryption needed for MAC verification

MAC-then-Encrypt

❌ Enc(K1, M || MAC(K2, M)) → only the ciphertext is sent
same plaintext → same MAC → encrypt → diff
↳ Bob has to decrypt first → Decryption oracle

⇒ separate keys → if K2 is known → attacker cannot recover M
✅ use separate keys.

Hash & MAC → no non-repudiation.

MAC-then-Encrypt ⇒ "lucky 13 attack"

---

uuid → universally unique identifier. → nobody can generate the same
↳ 128 bit binary
↳ converted to hex

$\frac{1}{2^{128}}$

enough randomness needed

id/key/IV

Pseudorandom Number Generators (PRNGs)

random looking o/p ⇒ not distinguishable from truly random in polynomial time.

entropy ⇒ measure of disorderedness.

random & unpredictable

equally likely outcomes → high entropy source

gaussian → clustered

uniform → equally likely

XZ utils backdoor → part of linux kernel.

RCE vulnerability.

many poor src + 1 good src = good entropy

true randomness ⇒ expensive + slow.

single src → biased.

CT

slide (2-5) + 6 (tentative)

PRNG → DRBG

truly random seed

properties { connectness, efficiency, security

PRNG

measure of randomness ⇒ entropy

entropy ↓ predictable

↳ measured in bits.

real life distributions → poisson, exponential, not generally uniform..

overall target → uniform

add a bad source → still has the entropy ⇒ adds something.

lowest value: 0

same seed → randomness exhausts sooner or later

↳ output starts to repeat

↳ reseed.

fixed seed → one-to-one map ⇒ pseudo-random numbers

* adding a new source will never reduce the entropy.

(s) bit seed ⇒ 2^s

s-bit seed ⇒ s-bit output ⇒ one-to-one map

s-bit seed ⇒ 2s r ⇒ 2^{2s} ⇒ not truly random.

* computationally indistinguishable → secure PRNG

Rollback resistance

attacker learns the internal PRNG state

↳ still won't be able to guess past outputs.

(backtracking resistance)

forward..

HMAC ⇒ $state_{t+1} = HMAC(state_t)$

↳ provides rollback resistance as HMAC is a one-way function.

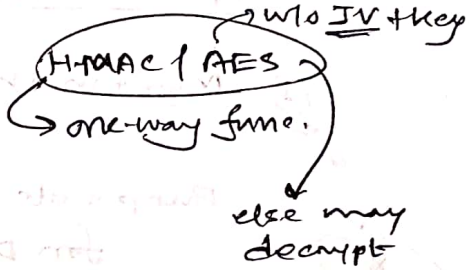
CTR-DRBG

AES-CTR ⇒ generates pseudo random bits by applying AES.

deterministic random bit generator. ↳ apply AES $\frac{m}{128}$ times.

HMAC-DRBG

↳ idea only, details not needed.
high-level diagram.



UUID

enough randomness, prob of generating the same number ≈ 0

↑ 128-bit unique values

Stream cipher eg AES-CTR

↳ pseudorandom number as the key to one-time pad.

n-bit counter ⇒ at most use upto n/2 length

exceed ⇒ new nonce.

continually generates as the data arrives.

Diffie-Hellman Key Exchange ⇒ math

one-way func / trap-door func → eg. mixing colors

↳ discrete logarithm problem.

g ⇒ generator → raise to diff exp & mod p

↳ covers all numbers from 1 to p-1.

a, b ⇒ random large numbers

$1 \leq a, b \leq p-2$ → no other conditions.

$g = 2$ $p = 131$

$g = 3$ → repeats for $n = 115, 117$

p → must be prime → else patterns start to appear.

ephemerality → later

1. Man-in-the-Middle Attack ⇒ vulnerable to this. (no authentication)

Burp suite does exactly the same thing just not for Diffie-Hellman..

limitation

2. key exchange requires both of them to be online at the same time

2 ways

① confirming from the user ⇒ Leap of faith

TF

$p = 353$

$g = 3$

$a = 97$

$b = 233$

public key: $\{g^a \bmod p, g^b \bmod p\}$

$g^{ab} \bmod p$ → shared secret

$g^a \bmod p = 3^{97} \bmod 353$ (97 = 64+32+1)

$3^1 \bmod 353 = 3$

$3^2 = 9$

$3^4 = 81$ (sq+mod)

$3^8 = 207$

$3^{16} = 136$

$3^{32} = 140$

$3^{64} = 185$

81

64+16+1

$3^{64} \cdot 3^{32} \cdot 3^1 \bmod 353$

$(140)(185)(3) = \ldots$

$= 40$

31/08/2026

Eve gets: g^a, g^b, g, p → cannot reconstruct..

$p = 7$

$g = 3$

$3^0 \bmod 7 \equiv 1$

$3^1 \equiv 3$

$3^2 \equiv 2$ (product)

$3^3 \equiv 6$

$3^4 \equiv 4$

$3^5 \equiv 5$

$3^6 \equiv 1$

$3^7 \equiv 3$

$3^8 \equiv 2$

pattern

generations will produce such patterns over $[1, p-1]$

uniformly distributed

↳ will touch every number.

period $= p-1$

$\text{ord}_7(3) = 6$ order of 3 wrt 7 = 6

Euler's theorem

Euler's totient theorem:

$$g^{\varphi(p)} \bmod p \equiv 1$$

$\varphi(p)$ → How many numbers are co-prime to p up until $p-1$ (inclusive)

if p is a prime → $(p-1)$

$\varphi(6) = 2$ $\{1,5\}$
$\varphi(7) = 6$ $[1-6]$

if g is a generator then raising it to $\varphi(p) \equiv 1 \bmod p$

first 1 at 3^0
second 1 at 36
} period $= p-1$

$p=7$
$g=2$ ⟶ not a generator ⟶ 1 to $p-1$ all must appear.

$2^0 \bmod 7 \equiv 1$
$2^1 \bmod 7 \equiv 2$
$2^2 ____ \equiv 4$
] 3,5,6 do not appear

$2^3 ____ \equiv 1$
$2^4 ____ \equiv 2$
$2^5 ____ \equiv 4$
]

⋮

Q. why do we want to use generators?

shared secret $= g^{ab} \bmod p$ ⇒ g not a generator → small subset
↳ easy to break
↳ entropy ↓

④ $g = 3$
$p = 353$
$a = 97$
$b = 233$

$A \equiv g^a \bmod p$
$B \equiv g^b \bmod p$ } public key

$g^{ab} \bmod p$ } shared secret

$A \equiv 40$ [fast modular exponentiation]

$B \equiv 248$ (check)

$g^{ab} \bmod p$ — 3 ways

- $3^{97 \times 233} \bmod p$ → ✗
- $(A)^{233} \bmod p$ ✗
- $(B)^{97} \bmod p$ → pick the lowest exponent

Elliptic-curve Diffie-Hellman → details → ✗

3072-bit DHE is about as secure as 384-bit ECDHE

Public Key Cryptography

↳ 2 keys for all
  ↳ public: known to all
  ↳ private:

msg → integers

* no shared secret needed
* slower than symmetric key cryptography

The Missing Semester

ssh → keygen → public, private/secret

M, e → numbers

Dec(SK, Enc(PK, M)) = M

* should be correct & reasonably fast.

CT slide : 26

05/09/2026

public key cryptography ⇒ messages are numbers → integers
{no scope for binary representation}

El Gamal Encryption

Taher Elgamal ⇒ father of SSL.

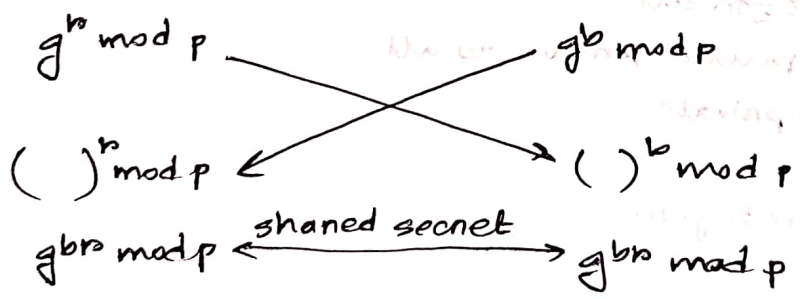
g, p } public base

Alice

secret: a

Bob

secret: b (private key)



~~DH~~ DH key Exchange

* cannot send messages ⇒ random secret
* both need to be random

ElGamal KeyGen

Bob → (b) → private key

$B = g^b \bmod p$ → public key → used for encryption

Enc(B, M)

Alice generates random b, $R = g^b \bmod p$ ⇒ doesn't send this to Bob

g, P } public base

Alice

secret: b

$R \equiv g^b \bmod p$

Bob

secret: b (private key)

$B \equiv g^b \bmod p$ (public key)

Alice derives the shared secret by multiplies by M

$M \times (B)^b \bmod p$

$(R, M * B^b)$

C_1 C_2

pair of ciphertexts

ciphertext → $(g^b, M * g^{br})$

similar to shared secret derivation → $(\ )^b \to g^{br}$

unit p

Modular inverse → g^{-br}

$*$

Dec (b, C_1, C_2) $\frac{C_1, C_2, b}{}$

M → can be anything

↳ can send messages

problem 1 solved

$M * g^{br} * g^{-br} \bmod p$

$\equiv M \bmod p$

⇒ both needn't be online

DH → sent C_1 beforehand ⇒ needed both to be online

Security

attacker has

~~g, b, g^b, ~~

Discrete logarithm problem
given g^b, cannot find b.

g, p, g^a, g^b, $M * g^{ab}$ → cannot recover M from these

⇒ need either a or b

$a \rightarrow (g^b)^a$

$b \rightarrow (g^a)^b$

Issues

⊛ not IND-CPA secure?

adversary (Eve) can send $M_0 = 0, M_1 \neq 0$ → non-zero

↓ always encrypted to 0

easily distinguishable

may use padding

⊛ may tampen with the msg → any multiple of M

$(c_1, \overbrace{k * c_2}^{c_2'})$ → Dec ⇒ kM

④ $g=2, p=11, b=4, M=7, r=3$

$B \equiv g^b \bmod p \equiv 2^4 \bmod 11 \equiv 5 \Rightarrow$ Bob's public key

$R \equiv g^r \bmod p \equiv 2^3 \bmod 11 \equiv 8 \Rightarrow c_1$

$B^r \equiv 5^3 \bmod 11 \equiv 4$

$M * B^r \equiv 7 \times 4 \bmod 11 \equiv 6$ $\Big\}$ $\boxed{7} * (5)^3 \bmod 11 \equiv 6$

$(8, 6)$

decrypt $\Rightarrow$ $8^4 \bmod 11 \equiv 8^2 \times 8^2 \bmod 11 \equiv 4$

$g^{br} \equiv 4 \xrightarrow{\text{modular inverse}} 3 \xrightarrow{* c_2} (3*6) \bmod 11 \equiv \boxed{7}$

$4x \bmod 11 \equiv 1$ ~~$4 = 12$ [illegible]~~

$4x + 11y = 1 \rightarrow x = 3$

M must be less than p → else clipped due to modulo

$12 \bmod 4 = 1$

huge prime numbers taken at a time 4KB msg - -1

Exercise

Mallory.

hill-additive → watch problem

block cipher → diagram problem

DHKE → problem

others → 99

RSA Encryption

KeyGen():
* pick two large primes randomly p & q
  Miller Rabin...

N = pq

choose e relatively prime to (p-1)(q-1)
$2 < e < (p-1)(q-1)$

modular inverse → $d = e^{-1} \bmod (p-1)(q-1)$

public key: N, e
private key: d

$\varphi(N)$ Euler's totient function
↳ # numbers from (1..N-1) relatively prime to N

$\varphi(p) = p-1$ (p → prime)

$\varphi(P_1 \cdot P_2) = (P_1-1)(P_2-1)$ (P_1 → prime, P_2 → prime)

Enc(e, N, M)

$M < N \Rightarrow$ mod N

$e \perp (p-1)(q-1)$ (e coprime to)

$c = M^e \bmod N$

public key $\{N = p \cdot q$, $e \perp (p-1)(q-1)$ ← $\varphi(N)$

secret key $\{d = e^{-1} \bmod (p-1)(q-1)$ ← $\varphi(N)$

$c = M^e \bmod N$ ← Encryption

$M = c^d \bmod N$ ← Decryption

correctness: $Dec(d, Enc(e, N, M)) = (M^e)^d = M \bmod N$

$$\boxed{M^{ed} \equiv M \bmod N}$$

(*) $p=5, q=11$

$N=55$

$\varphi(N) = 4 \times 10 = 40$ → sanity check

$e=3 \Rightarrow$ given ↙

$m < 55$

$3 \cdot d \equiv 1 \bmod 40 \rightarrow d = 27$

$3x + 40y = 1$

$40 = 3 \cdot 13 + 1$

$1 = 3(-13) + 40(1)$

$x = -13 + 40 = (27)$ minimum +ve value

$m=10$

$c = 10^3 \bmod 55 = 10$

$m = 11 \Rightarrow c = 11^3 \bmod 55 = 11$ } rare matches

$m = 12 \Rightarrow c = 12^3 \bmod 55 = 23$

$23^{27} \bmod 55$

$\equiv (23 \cdot 34) \bmod 55$

$\equiv 12$

$27 = 16 + 8 + 2 + 1$

$23^1 \bmod 55 \equiv 23$

$23^2 \bmod 55 \equiv 34$

$23^4 \bmod 55 \equiv 1$

$23^8 \bmod 55 \equiv 1$

$23^{16} \bmod 55 \equiv 1$

55 | 1331 | 110 231

correctness → proof

⊛ Chinese Remainder Thm (CRT): Lemma

p, q prime (↔ coprime), $x \equiv y \bmod p$, $x \equiv y \bmod q$ $\Big\}$ $x \equiv y \bmod pq$

same congruence wrt pq

⊛ Fermat's little theorem (F.L.T):

$a^{\varphi(p)} \equiv 1 \bmod p$ → Euler's thm

if p is prime, $a^{p-1} \equiv 1 \bmod p$ → FLT

$ed \equiv 1 \bmod (p-1)(q-1)$

$ed \equiv 1 \bmod \varphi(N)$ → $(p-1)(q-1)$

$(ed-1)$ = multiple of $\varphi(N)$

$0 \to 1$

$[1, (p-1)]$ all occurs $\quad p-1 \to 1$

$M^{ed} \equiv M^{ed-1} M \bmod p$

$\equiv M^{k \cdot \varphi(N)} M \bmod p$

$\equiv M^{\text{multiple of } (p-1)} \cdot M \bmod p$

FLT (1)

$\equiv M \bmod p$

$M^{ed} \equiv M \bmod N$

congruence

Med is congruent to M modulo N

$M^{ed} \equiv M \bmod q$

CRT

$M^{ed} \equiv M \bmod pq$

$a^{(p-1)k} \equiv (1)^k \bmod p$

may modify the algo $(p-1)(q-1)(r-1)$ → 3 primes

security : depends on private key, d
↳ need to know (pq)

if N can be factored ⇒ break RSA need to factor N

4000 bit product → cannot solve in polynomial time

$p, q = 2k, 2k$ bits

3 primes ⇒ 1.3k bits
smaller primes
↳ easier to break

⇒ 2 primes present the most difficult case
↳ hence we use 2 of them.

RSA → deterministic → not IND-CPA secure.

* $m << N$
$e \sim 3$ → small e
$m^3 << N$

$c \equiv m^3 \bmod N$ ⇒ no chance to wrap around

$m = \sqrt[3]{c}$ → recover N

ensure the modulo operation works at least a few times.

* timing attacks are also possible.

modifications to prevent these ..

Hybrid encryption:

$K \Rightarrow 128, 256$ bits.

share the key using asymmetric encryption

share the data " symmetric "

Digital Signatures

① how to ensure public key sharing itself is secure?

encrypt using private key and decrypt with the public one.

nobody else can encrypt — anyone may verify.

KeyGen()

Sign(SK, M) → encrypt with the secret key

Verify(PK, M, sig) → {0, 1}

EU-CPA

large message ⇒ hash the msg → assumption: collisions are rare.

deterministic

↓ sign

RSA signature

Sign: $S \equiv M^d \bmod N$

Verify: $M = S^e \bmod N$

} actually done on the msg hash.

DSA signature → not in syllabus.

⊛ certificates:

digital signature ⇒ only one person can encrypt
↳ using private key

⇒ any public key method is vulnerable to MITM.
sharing the public key ⇒ use digital signature

Bob sends ⇒ { PK_B, sign = E (PK_B, SK_B) }
↳ secret
↳ public

Mallory can change both.
↳ still vulnerable.

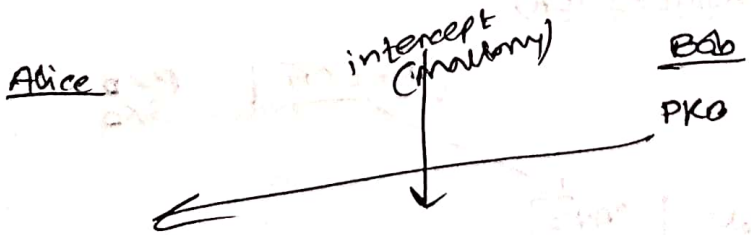
Trust anchor ⇒ the third party we rely on / trust

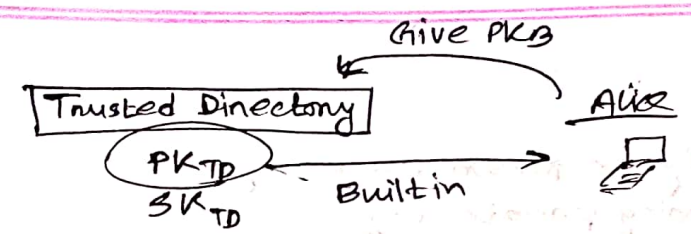
Trust on first use ⇒ Leap of faith ⇒ ssh:
↳ yes/No

↳ ask the user the first time.
next time ⇒ user is responsible if anything goes wrong

assumption ⇒ MITM is unlikely the first time.

root of trust ⇒ certificate → signed endorsement of someone's public key.





certificate: { Bob's public key is PK_B }$_{SK_{TD}}$

endorsement

TD is responsible for this verification

Alice blindly trusts TD

↳ decrypt ⇒ PK_B

PK_{TD} ⇒ built in the computers with browsers...

not fetched (to MITM)

SK-1 ⇒ encrypted with secret key.

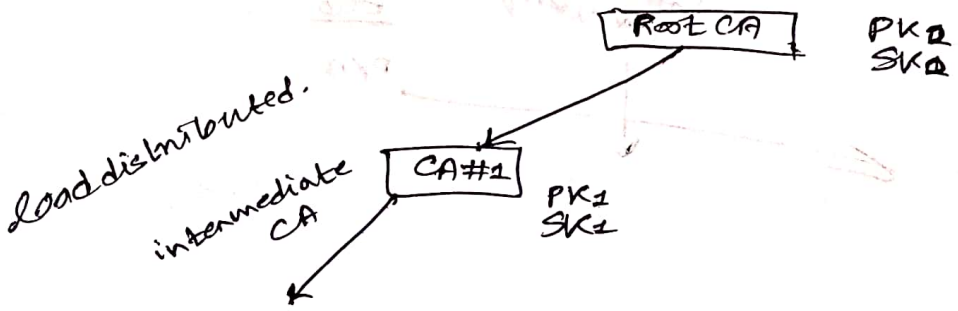
TD cannot support everybody → Scalability ⇒ problem 1

TD down ⇒ communication halted

TD hacked ⇒ compromised → none to trust } single point of failure.

address the scalability issue

Certificate Authorities (CA)



{CA#1's public key PK1}SK_0 → root CA signs this
→ can trust CA # 1

delegate the trust → hierarchical model of CA ⇒ currently used model.

{msg, {H(CA#1's public key PK1)}SK_0}

↳ plaintext
↳ may be lengthy → do not encrypt → only encrypt the hash.

* 160-170 root CAs → built-in the OS/browsers...

if CA # 4 does not have PK_3 ⇒ ask from the upper layer...
↳ finally → root

* so many root CAs trusted → anyone may be compromised. } issue
if only one of the CAs return the key ⇒ accept

* certificate authority may mess up.

↳ soln

① expiration dates ⇒ wrong certificate → limited damage

↳ caveat ⇒ renew the certificate
security ∝ $\frac{1}{\text{usability}}$

moodle ⇒ connection is secure → https (locks)
↳ ① all comms are encrypted
↳ ⑪ the certificate is still valid.

using moodle's PUK.
given by CA } verified using certificate

② Revoked certificates → announce

CRL → signed by CA

↳ large lists

↳ w/o latest list → ✗

} not used (as much) anymore, expiration ↑

hierarchical trust → tree

graph ⇒ web of trust

openPGP → disaster

} not used

CSE-405 13/09/2026

Password Hashing

cryptographic hash
- ↳ fast
- ↳ collision resistant

* stone pwd in a file → bad
* encrypt before storing
  - ↳ check → compare ciphertext } where do we keep the keys? (bad).

need one-way-ness ⇒ hashing
- → deterministic } properties
- → one-way

Attacks

* assume the hacker has access to the DB
  - same pwd → same hash ⇒ not IND-CPA secure

use nonce → user specific
- ↳ store this in plaintext

dictionary attack
- ↳ common pwd hash
- ↳ compare against the db using

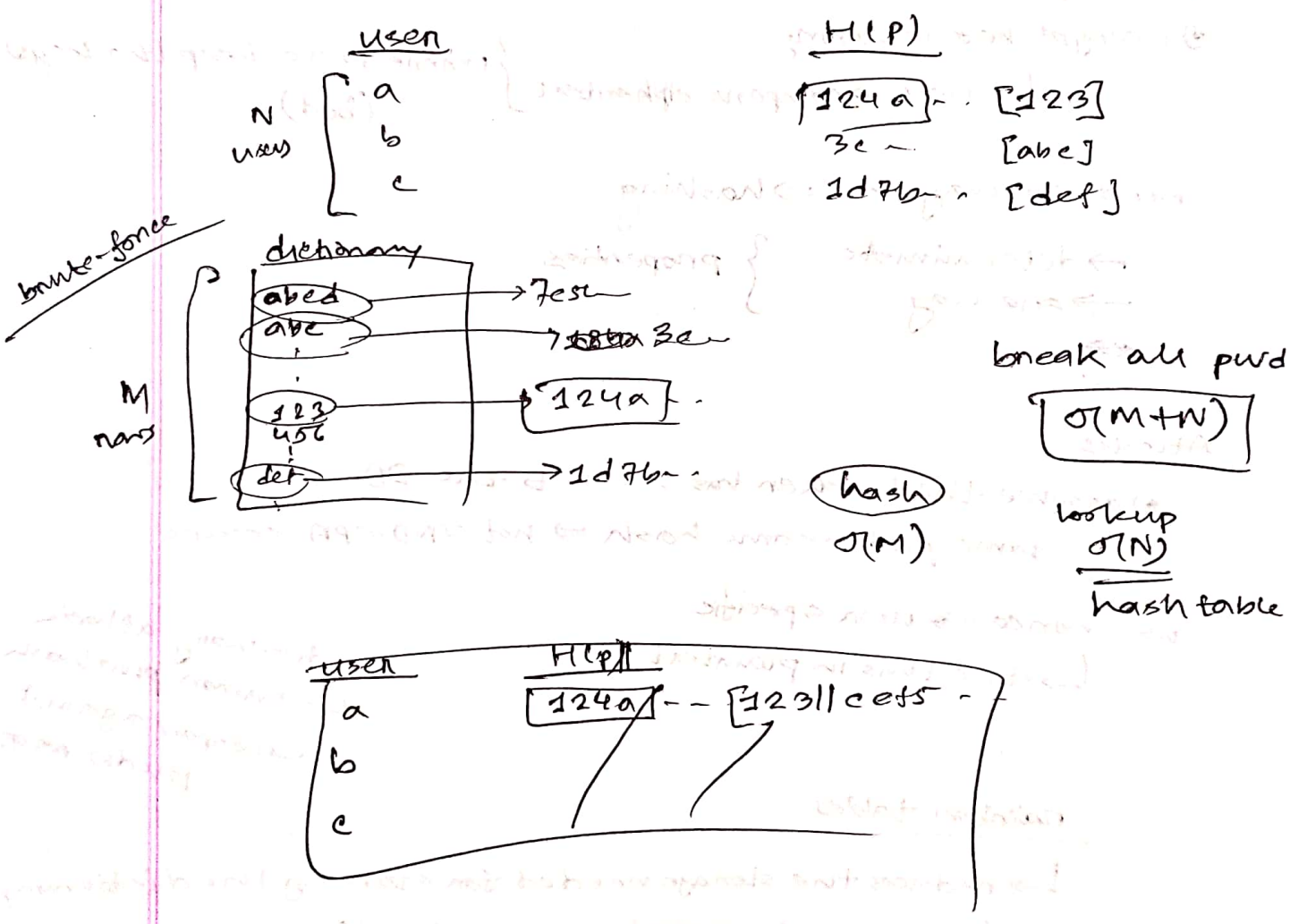
rainbow tables

↳ reduces the storage needed for storing the dictionary
- ↳ increased compute → trade-off

Nonce → salt ⇒ random number

username, salt, H(password||salt) → bcrypt (JS)

8 char, lowercase → 26^8 pwds



user	H(p‖salt)	p (salt → also in plaintext)
a	765f...	1231‖cef5...
b	1ebc...	abe‖1657...
c	4dfc...	def‖74da...

$O(MN)$ ⇒ significantly tough.
↳ not feasible.

iterate through all pwds for each salt

user ↓ # salt ↓ ⇒ easily crack the pwds.

* modern GPUs can compute 10k / 20k times more hashes than CPUs per second.
↳ salt gets less effective

SOH ⇒ slower hashes

offline attack: attkr has all the data → downloaded from the server
↳ tough to prevent
→ use slower hash

online attack: interacts and tries to brute force the server
↳ rate limit

Traffic Analysis
Side channel
} defn, difference ⇒ (99)

eg Alice sends "Hi, Bob!" msg every morning
↳ analyze → may be greetings
(communication pattern analysis)
eg predict upcoming protests.
encrypted pkt freq analysis

side channel → time/power consumption → spike.

Nothing-Up-My-Sleeve numbers

constants in cryptography → disclose the numbers
↳ how these were generated

cannot hide these

Introduction to the Web

Internet → how data is transported
Web → frontend ↔ backend

URL

http vs https

URL → # fragment → client side (→ scroll to the text.)

port, path, query → handled by the server

domains can even be : username@domain
but not used.

URL escaping

?, #, L → encode → %hex

↳ makes scanning for malicious URLs harder

Security on the Web

risk #1: Web servers should be protected from unauthorized access
↳ server-side security

risk#2: a malicious website should not be able to damage our computer
↳ sandboxing

risk# 3: a malicious website should not be able to tampen with our interactions with other websites
↳ same-origin policy.
↳ enforced by the browser (frontend)

same-origin: if protocol, domain, port all match
→ serially ⇒ short ckt check

default port → 80

exception1 CORS ⇒ cross-origin resource sharing

exception2 display images → URL → any

exception3 iframe → embed ⇒ embedde website JS

Tracking Pixel

ThunderBird → embed HTML

gmail: first req → local IP
next → Google server cache

Cookies & CSRF

log in → session token → stored as cookie

theme data →
analytics → focused portions } saved locally in the browser

http ⇒ stateless protocol.

name-value pair.

domain
path → attached to each cookie

Secure → enabled ⇒ only send the cookies if the website has https enabled

HttpOnly → prevent cookie stealing
JS cannot access the cookies
document.getCookies()...
http → get cookie

⊛ cookies are created by the server

cookie policy → any server can create a cookie for another one → browser accepts/rejects based on the policy.

Browser

www.malicious.com

Browser — www.malicious.com

Set-Cookie: Domain=www.malicious.com, ...

same domain → accept

Browser — mail.google.com

Set-Cookie: Domain=mail.google.com, ...

google.com

setting

sub-domain can set the cookie for the upper level domain

but not .com → TLD

↳ too many.

any suffix → except TLD

sending

cookie domain → right-align with the server

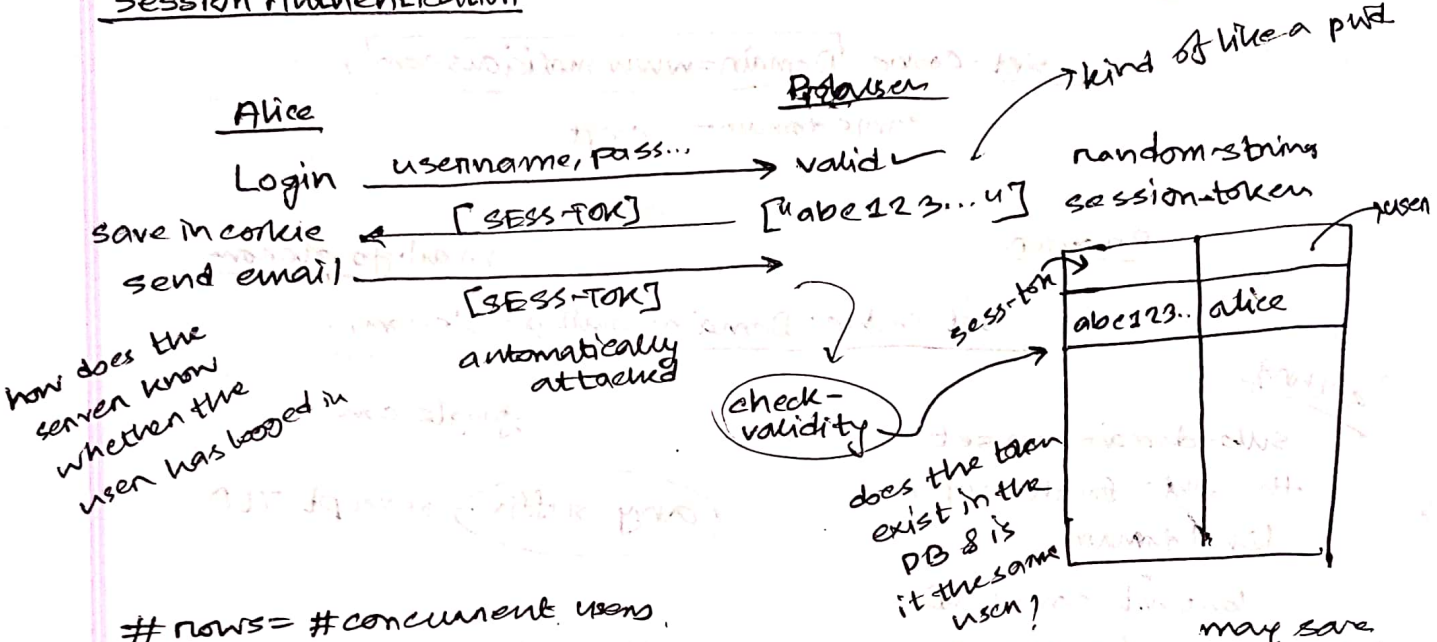
" path → left-align " " "

suffix prefix

etc

Q server URLs, cookies → which ones may be sent?

Session Authentication



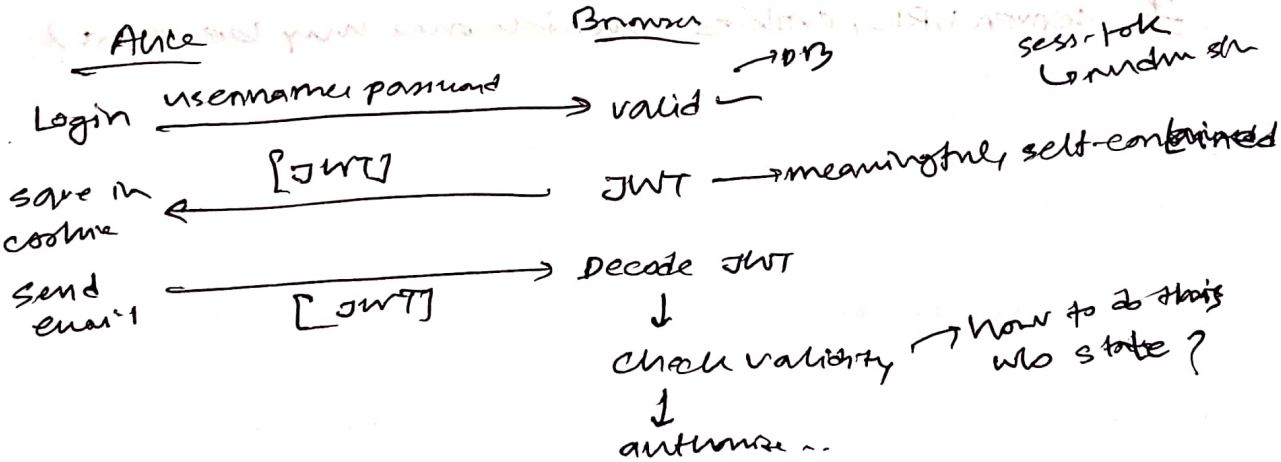
rows = # concurrent users.

drawback

↳ need to maintain separate states for each concurrent user

JSON Web Token

JWT ⟹ stateless ⟹ server doesn't save anything



JWT

signature ← server sends the hash of the header & payload here.

only server can do this

HMAC → uses a secret key

header . payload . ~~data~~ ⇒ 3 parts

↓ metadata

```
{
  "alg": "HS256",   → HMAC-SHA-256
  "type": "JWT"
}
```

actual info

```
{
  "user": "alice",
  "role": "non-admin",        may have "permissions": [...]
  "iat": Sept 19, 2026...     ← issued at
  "expires": Sept 30, 2026... ⇒ validity.
}
```

today > expires ⇒ discard...

raw json → base 64 encoding → concat → actual JWT

check user → match

role → abilities.

validity ⇒ what if alice uses "role": "admin"

signature ⇒ HMAC(header||payload, K) → secret key.

session/jwt secret

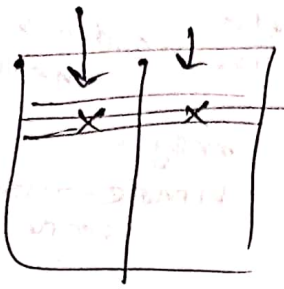
→ user cannot generate the signature.

⇒ server generates the hash & matches it with the signature.

advantage ⇒ no server side state

drawback ⇒ log out ⇒ the JWT is not invalidated. → not true logout

anybody may use the JWT later on



sess-tok ⇒ logout → remove from DB
subsequent req ⇒ needs to log in.

⑨ comparison, benefits, drawbacks: session token vs jwt

session token or jwt → stolen → problematic

secure: true ⇒ only https

Httponly: cannot access via JS ⇒ set to true
↳ NULL
↳ xss

session auth diagram

some old systems

legacy systems

↳ send a random invalid session-id
whenever someone visits the page
↳ login → mark the id as valid

problem ⇒ the session-id is stored at the client.
↳ sent along with username+pwd.

bjis.buet.ac.bd/..

↓

[bjis.buet.ac.bd/ sess-id = abc123...]

→ xyz123 ⇒ set by the attacker

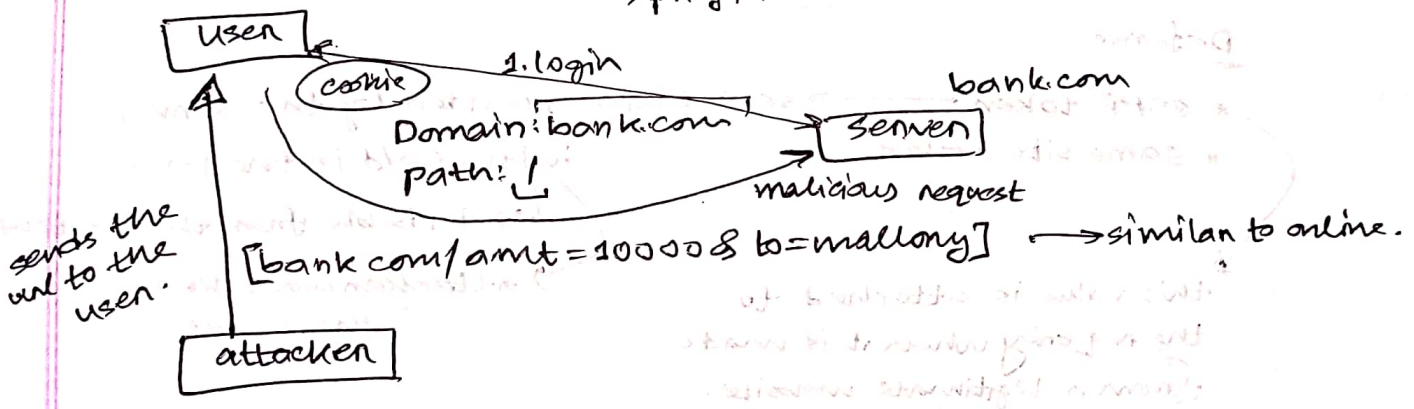
use the url ⊗ constructed by attkr → Session fixation attack

↳ the id becomes valid
↳ username+pwd
} attkr can pose as the user now

this problem only exists when the session-id is sent beforehand

CSRF

bank.com/amt=100&to=bob
↳ prefix



⊗ how to trick the user?

CSE-405 20/09/2024

XSS, CSRF → similar → (?) ⇒ differences

↳ execute JS → malicious req

how to trick the victim? ⇒ CSRF

GET

POST ⇒ execute JS on create a form

↳ XSS for CSRF

Defense

* CSRF token ⟶ secret value provided by the server
  hidden field in the form
  ↳ not visible from the frontend
  ↳ attacker won't know the secret
  ↳ this value is attached to the req only when it is made from a legitimate website.
* same site cookie

↳ attacker doesn't know the value ⇒ stored at the frontend & the legitimate website

optional field

referer header ⇒ set by the browser → attacker cannot control this w/o MITM
↳ which webpage made the request.

same-site ⇒ referer header matches the URL
↳ blocks cross-site requests

→ may leak private info. ⇒ which websites the user visited

→ depends on the server's policy.

SameSite cookie attribute

strict ⇒ domain must match exactly to the origin

lax ⇒ allows cookie attachment from cross site requests
needed for some specific scenarios –
eg- google search ⇒ login not needed when clicking on a website

Q ⇒ scenario → what might be the reason?
samesite=strict.

XSS

same-site vs same-origin

solution ⇒ sanitization → use libraries.

similar to sql injection
↳ parameterized query

② types
* stored xss → persistent → server side vulnerability
* reflected xss

Q scripts should not be able to access cookies → httpOnly

Q attacker → injected malicious script
↳ steal session token, JWT
↳ still gets nothing → why? } httpOnly: true

reflected xss ⇒ url → script within the query.
victim makes a req with injected JS. → eg from some ad.

↳ no direct comm. betn the attacker & the server.

stored → direct comm.

CSRF vs reflect XSS

- client side execution
- executes a script
  - may or may not attach a cookie
- attaches a cookie
  - unintended req
- server-side execution.

defense

- sanitization. `<` → `<` `>` → `>` `"` → `"`
  - eg string escaping

Golang

{{ }}

automatically escapes.

- content security policy (CSP)
  - client side defense → browsers.

UI Attacks → reason: browsers trusts the user's actions.

- clickjacking → eg. invisible iframe, temporal attack, cursorjacking
- phishing
  - impersonate some website to collect personal info
  - homograph attack
    - similar looking urls

defense ⇒ enforce visual integrity, temporal, require confirmation, frame-busting

2FA

CSE-405

21/09/2026

Subverting 2FA: Relay Attacks

only one comm both server & victim

vs MITM
↳ no comm both server & victim

SQL injection & CAPTCHA

UNION → may work on multiple tables
ensure the col names match & order

Blind SQL injection

↳ no feedback
↳ may be pwd mismatch
↳ may be wrong syntax
} different time taken
timing attack

defense

* sanitization
* prepared statement → not part of the SQL standard.
(?) (DOS) ⇒ needs to be compiled
⇒ vulnerable query compiled.
↳ compiled with the placeholder beforehand.
(treated as data)

command injection

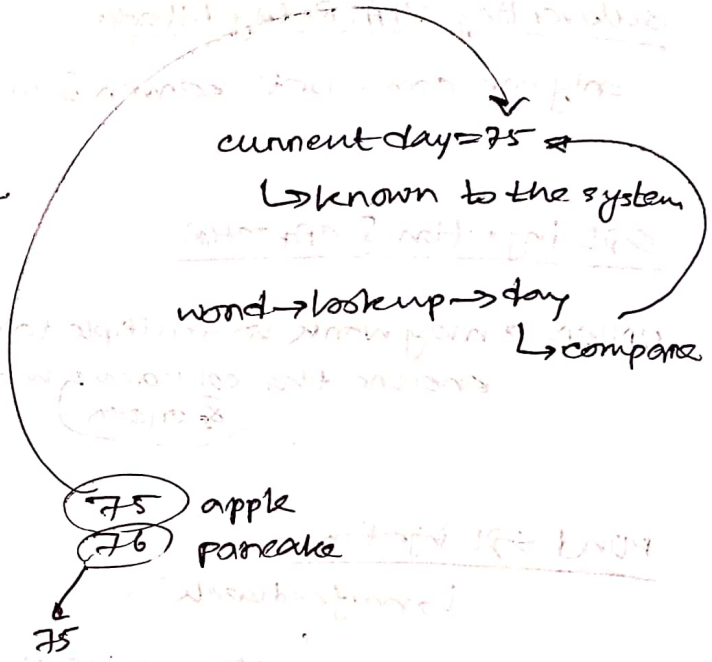
↳ sanitize
↳ safe API

CAPTCHA → now used to prevent DOS attacks.
not that effective anymore

TF

wordle

day	word
1	
2	
3	a
⋮	
365	



find - -

SELECT day FROM answers WHERE word='__'

UNION -- day-1 - -

CT3

1. Encrypt then MAC → MAC(K2, Enc(K1, M))
3. invertible mdm 26
8. PRNG ⇒ 32 bits → 2^{32}

$C1 = E(P1 \oplus IV1) \oplus IV2$
$C2 = E(P2 \oplus C1)$
$C3 = E(P3 \oplus C2)$

$P1 = D(C1 \oplus IV2) \oplus IV1$
$P2 = D(C2) \oplus C1$
$P3 = D(C3) \oplus C2$

$0 = D(C1 \oplus IV2) \oplus IV1 \oplus P1$

$P1' = D(C1 \oplus IV2) \oplus IV1 \oplus P1 \oplus P1'$ $\quad IV1' = IV1 \oplus P1 \oplus P1'$

$P2' = D(C2) \oplus C1 \oplus P2 \oplus P2'$ $\quad C1' = C1 \oplus P2 \oplus P2'$

$C2' = C2$

$C1 \oplus IV2 = C1' \oplus IV2'$

$\Rightarrow IV2' = C1 \oplus C1' \oplus IV2 \oplus C1'$

$= C1 \oplus IV2 \oplus C1 \oplus P2 \oplus P2'$

$\boxed{IV2' = IV2 \oplus P2 \oplus P2'}$